Final Project Code

E. Morra, S. Miller

2023-11-30

R Markdown

Libraries

```r
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.3.2
```

```r
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.3.2

## Warning: package 'dplyr' was built under R version 4.3.2

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.4
## v forcats   1.0.0     v stringr   1.5.0
## v lubridate 1.9.2     v tibble    3.2.1
## v purrr     1.0.2     v tidyr     1.3.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(dplyr)
library(tidymodels)
```

```
## Warning: package 'tidymodels' was built under R version 4.3.2

## -- Attaching packages -------------------------------------- tidymodels 1.1.1 --
## v broom        1.0.5     v rsample      1.2.0
## v dials        1.2.0     v tune         1.1.2
## v infer        1.0.5     v workflows    1.1.3
## v modeldata    1.2.0     v workflowsets 1.0.1
## v parsnip      1.1.1     v yardstick    1.2.0
## v recipes      1.0.8

## Warning: package 'dials' was built under R version 4.3.2

## Warning: package 'infer' was built under R version 4.3.2

## Warning: package 'modeldata' was built under R version 4.3.2

## Warning: package 'parsnip' was built under R version 4.3.2

## Warning: package 'recipes' was built under R version 4.3.2

## Warning: package 'rsample' was built under R version 4.3.2

## Warning: package 'tune' was built under R version 4.3.2
```

```
## Warning: package 'workflows' was built under R version 4.3.2

## Warning: package 'workflowsets' was built under R version 4.3.2

## Warning: package 'yardstick' was built under R version 4.3.2

## -- Conflicts ------------------------------------------ tidymodels_conflicts() --
## x scales::discard() masks purrr::discard()
## x dplyr::filter()   masks stats::filter()
## x recipes::fixed()  masks stringr::fixed()
## x dplyr::lag()      masks stats::lag()
## x yardstick::spec() masks readr::spec()
## x recipes::step()   masks stats::step()
## * Use suppressPackageStartupMessages() to eliminate package startup messages
```

```r
library(gghighlight)
```

```
## Warning: package 'gghighlight' was built under R version 4.3.2
```

```r
library(knitr)
```

```
## Warning: package 'knitr' was built under R version 4.3.2
```

```r
set.seed(1234)
```

San Francisco Data Set Data Set has "San Francisco city employee salary data(gender column included)"

```r
#read in file
SF_dataset <- read.csv("SF_salary_data_gender.csv")
#data clean
SF_dataset <- SF_dataset[, !(colnames(SF_dataset) %in% c("Benefits", "TotalPayBenefits", "Status", "X"))
SF_dataset <- SF_dataset[SF_dataset$gender != "", ]
SFmales <- sum(SF_dataset$gender == "m")
SFfemales <- sum(SF_dataset$gender == "f")
print(colnames(SF_dataset))
```

```
##  [1] "Id"            "Employee.Name" "JobTitle"      "BasePay"
##  [5] "OvertimePay"   "OtherPay"      "TotalPay"      "Year"
##  [9] "Notes"         "Agency"        "gender"
```

```r
print(paste("The amount of males in the data set is is:", SFmales))
```

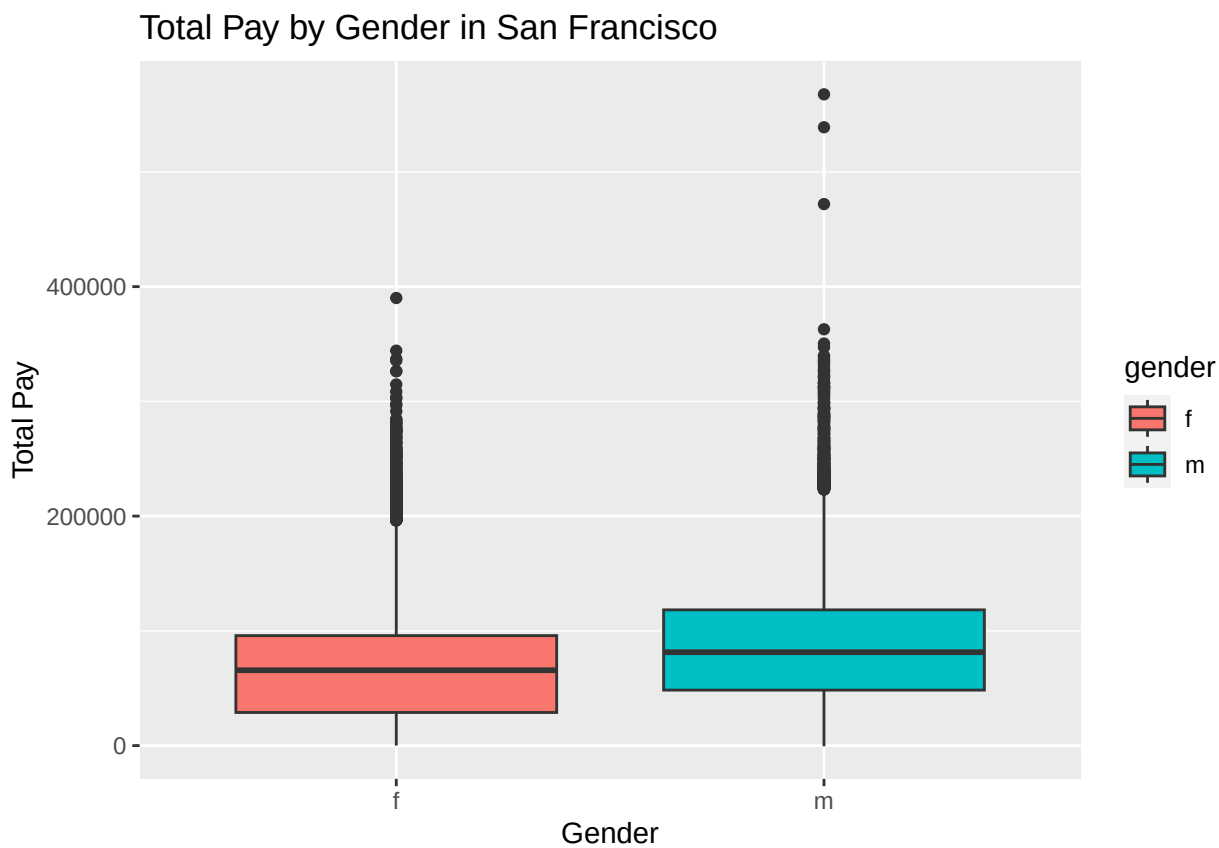
```
## [1] "The amount of males in the data set is is: 67820"
```

```r
print(paste("The amount of females in the data set is is:", SFfemales))
```

```
## [1] "The amount of females in the data set is is: 51349"
```

Analyze and visualize cleaned San Francisco Data Set Summary

```r
options(scipen = 600000)
ggplot(SF_dataset, aes(x = gender, y = TotalPay, fill = gender)) +
  geom_boxplot() +
  labs(title = "Total Pay by Gender in San Francisco",
       x = "Gender",
       y = "Total Pay")
```




```
mean_totalpay_f<- mean(SF_dataset$TotalPay[SF_dataset$gender == "f"], na.rm = TRUE)
mean_totalpay_f<- round(mean_totalpay_f, 2)

mean_totalpay_m<- mean(SF_dataset$TotalPay[SF_dataset$gender == "m"], na.rm = TRUE)
mean_totalpay_m<- round(mean_totalpay_m, 2)

print(paste("The mean total pay for females is:", mean_totalpay_f))
```

```
## [1] "The mean total pay for females is: 68448.29"
```

```
print(paste("The mean total pay for males is:", mean_totalpay_m))
```

```
## [1] "The mean total pay for males is: 83932.7"
```

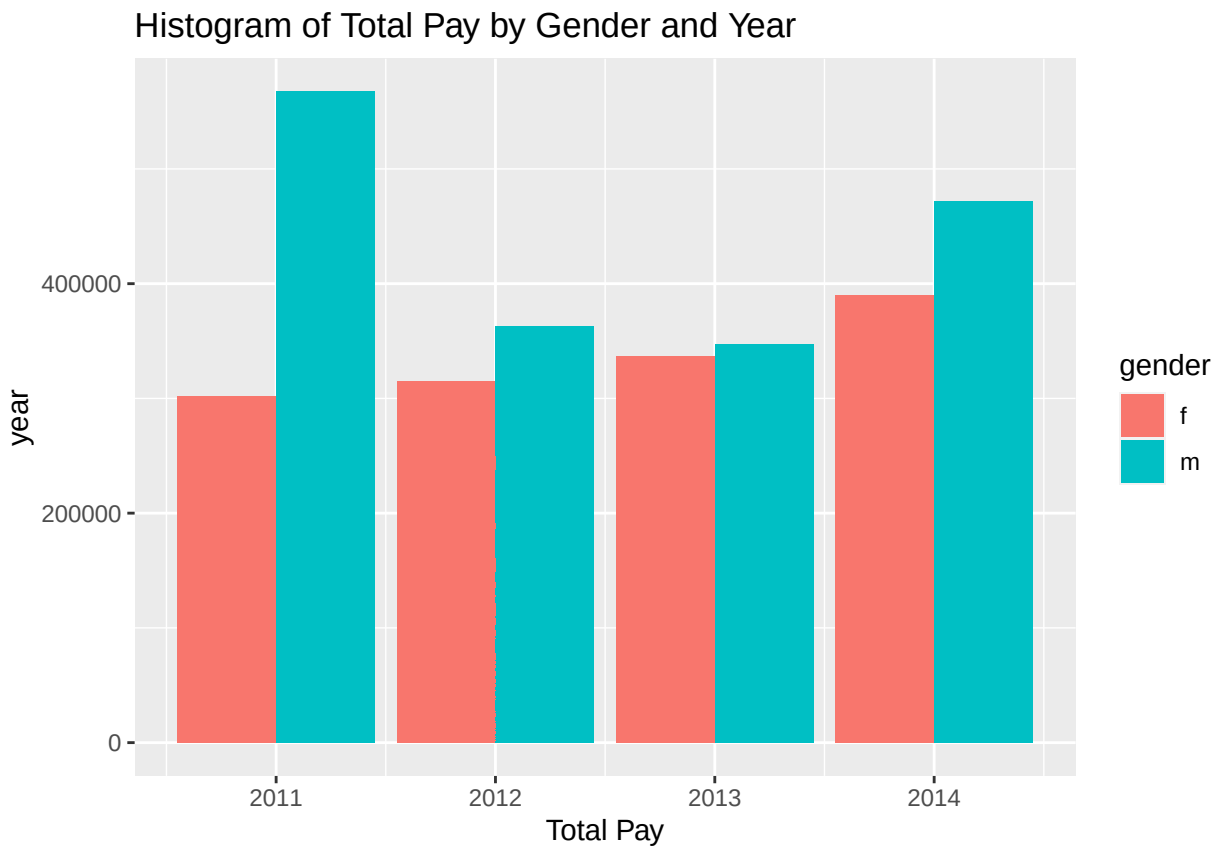
```
max_totalpay_f<- max(SF_dataset$TotalPay[SF_dataset$gender == "f"], na.rm = TRUE)
max_totalpay_f<- round(max_totalpay_f, 2)

max_totalpay_m<- max(SF_dataset$TotalPay[SF_dataset$gender == "m"], na.rm = TRUE)
max_totalpay_m<- round(max_totalpay_m, 2)

print(paste("The maximum total pay for females is:", max_totalpay_f))
```

```
## [1] "The maximum total pay for females is: 390111.98"
```

```
print(paste("The maximum total pay for males is:", max_totalpay_m))
```

```
## [1] "The maximum total pay for males is: 567595.43"
```

```r
ggplot(SF_dataset, aes(x = Year, y = TotalPay, fill = gender)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Histogram of Total Pay by Gender and Year",
       x = "Total Pay",
       y = "year")
```



Texas Government Salary Data Set

```r
#read in file
texas_gov <- read.csv("texas_salaries.csv")
#data clean
remove_cols <- c("AGY", "MI", "CLASS.CODE", "STATE.NUMBER", "duplicated",
                 "multiple_full_time_jobs", "combined_multiple_jobs", "summed_annual_salary",
                 "hide_from_search")
texas_gov<- texas_gov[, !(names(texas_gov) %in% remove_cols)]

texas_gov <- texas_gov[complete.cases(texas_gov), ]

print(unique(texas_gov$GENDER))
```

```
## [1] "MALE           " "FEMALE         "
```

```r
texasmales <- sum(texas_gov$GENDER == "MALE           ")
texasfemales <- sum(texas_gov$GENDER == "FEMALE         ")
texas_gov <- na.omit(texas_gov)

print(colnames(texas_gov))
```

```
##  [1] "AGENCY.NAME" "LAST.NAME"   "FIRST.NAME"  "CLASS.TITLE" "ETHNICITY"
##  [6] "GENDER"      "STATUS"      "EMPLOY.DATE" "HRLY.RATE"   "HRS.PER.WK"
## [11] "MONTHLY"     "ANNUAL"
```

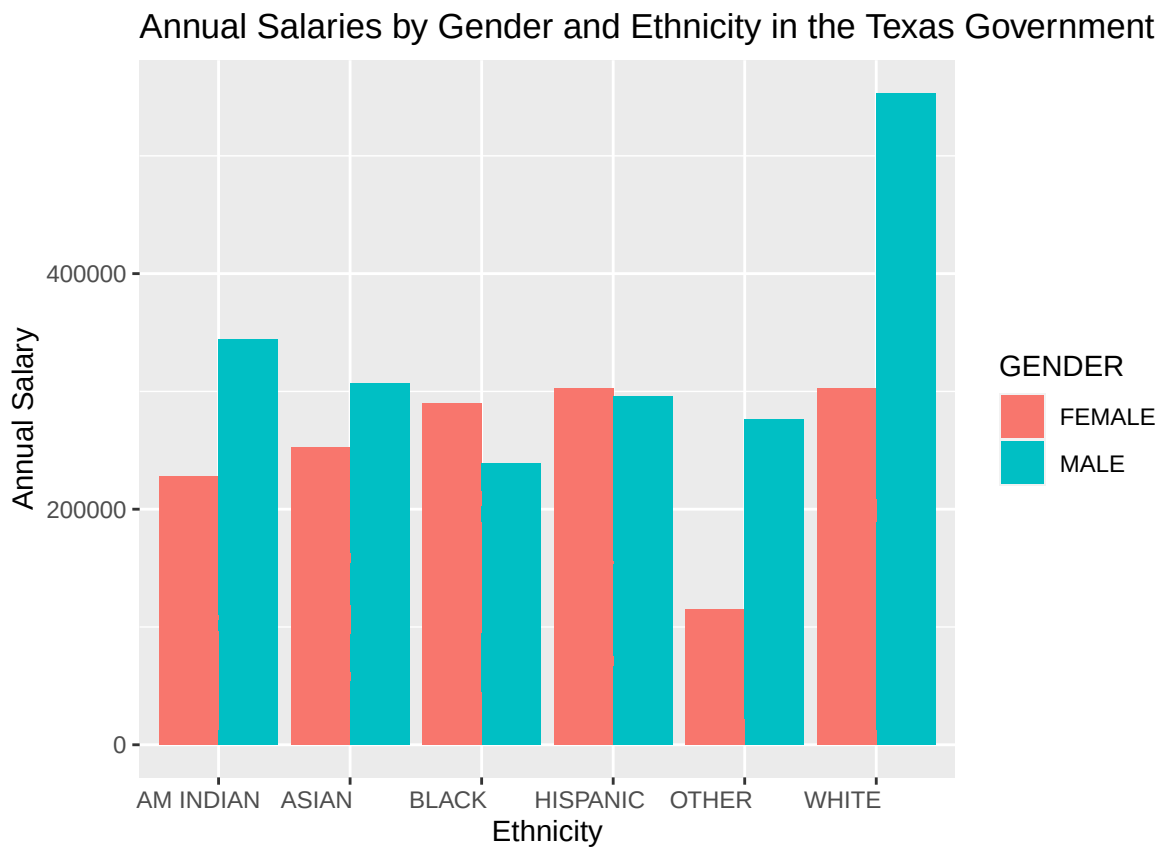
```
print(paste("The amount of males in the data set is is:", texasmales))
```

```
## [1] "The amount of males in the data set is is: 63254"
```

```
print(paste("The amount of females in the data set is is:", texasfemales))
```

```
## [1] "The amount of females in the data set is is: 84818"
```

```
ggplot(texas_gov, aes(x = ETHNICITY, y = ANNUAL, fill = GENDER)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Annual Salaries by Gender and Ethnicity in the Texas Government",
       x = "Ethnicity",
       y = "Annual Salary")
```



Multiple Linear Regression

```
print(unique(texas_gov$GENDER))
```

```
## [1] "MALE           " "FEMALE         "
```

```
print(unique(texas_gov$ETHNICITY))
```

```
## [1] "WHITE          " "BLACK          " "HISPANIC       " "OTHER          "
## [5] "AM INDIAN      " "ASIAN          "
```

```
texas_gov$gender_num <- as.integer(factor(texas_gov$GENDER, levels = c("MALE           ", "FEMALE
texas_gov$ethnicity_num <- as.integer(factor(texas_gov$ETHNICITY, levels = c("WHITE          ", "BLACK
```

```r
texas_annual <- lm(ANNUAL ~ gender_num + ethnicity_num, texas_gov)
names(texas_annual)
```

```
##  [1] "coefficients"  "residuals"     "effects"       "rank"
##  [5] "fitted.values" "assign"        "qr"            "df.residual"
##  [9] "xlevels"       "call"          "terms"         "model"
```

```r
coefficients(texas_annual)
```

```
##   (Intercept)    gender_num ethnicity_num
##     63634.164     -6551.231     -1467.545
```

```r
summary(texas_annual)
```

```
##
## Call:
## lm(formula = ANNUAL ~ gender_num + ethnicity_num, data = texas_gov)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -54575 -14737  -6015   8384 497885
##
## Coefficients:
##               Estimate Std. Error t value            Pr(>|t|)
## (Intercept)   63634.16     236.84  268.68 <0.0000000000000002 ***
## gender_num    -6551.23     129.56  -50.56 <0.0000000000000002 ***
## ethnicity_num -1467.54      57.73  -25.42 <0.0000000000000002 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 24630 on 148069 degrees of freedom
## Multiple R-squared:  0.02205,    Adjusted R-squared:  0.02204
## F-statistic:  1669 on 2 and 148069 DF,  p-value: < 0.00000000000000022
```

Testing / training - see lab 10

- Create an initial split

```r
tex_tt <- texas_gov[, -(1:4)]
tex_tt <- tex_tt[, -(4:7)]
tex_tt <- tex_tt[, -(5:6)]
# take away unneccesary vars
set.seed(1122)
texas_split <- initial_split(tex_tt) # prop = 3/4 by default
```

- Save training data

```r
texas_train <- training(texas_split)
dim(texas_train)
```

```
## [1] 111054      4
```

- Save testing data

```r
texas_test  <- testing(texas_split)
dim(texas_test)
```

```
## [1] 37018     4
```

```r
texas_mod <- linear_reg() %>%
  set_engine("lm")

texas_mod
```

Specify model

```
## Linear Regression Model Specification (regression)
##
## Computational engine: lm
```

```r
texas_rec <- recipe(ANNUAL ~ ., data = texas_train) %>%
  # remove zero variance predictors
  step_zv(all_predictors())
```

```r
texas_rec
```

Build recipe

```
##
## -- Recipe ----------------------------------------------------------------------
##
## -- Inputs
## Number of variables by role
## outcome:   1
## predictor: 3
##
## -- Operations
## * Zero variance filter on: all_predictors()
```

```r
texas_wflow <- workflow() %>%
  add_model(texas_mod) %>%
  add_recipe(texas_rec)
```

```r
texas_wflow
```

Build workflow

```
## == Workflow ====================================================================
## Preprocessor: Recipe
## Model: linear_reg()
##
## -- Preprocessor ----------------------------------------------------------------
## 1 Recipe Step
##
## * step_zv()
##
```

```
## -- Model ----------------------------------------------------------------------
## Linear Regression Model Specification (regression)
##
## Computational engine: lm
```

```
texas_fit <- texas_wflow %>%
  fit(data = texas_train)
```

```
tidy(texas_fit) %>%
  print(n = 12)
```

Fit model

```
## # A tibble: 17 x 5
##    term                                  estimate std.error statistic   p.value
##    <chr>                                    <dbl>     <dbl>     <dbl>     <dbl>
##  1 "(Intercept)"                           48584.      984.     49.4  0
##  2 "ETHNICITYASIAN          "              11276.     1060.     10.6  2.01e- 26
##  3 "ETHNICITYBLACK          "              -7206.      992.     -7.26 3.79e- 13
##  4 "ETHNICITYHISPANIC       "              -3926.      991.     -3.96 7.44e-  5
##  5 "ETHNICITYOTHER          "              -3131.     1386.     -2.26 2.39e-  2
##  6 "ETHNICITYWHITE          "               3496.      988.      3.54 4.00e-  4
##  7 "GENDERMALE           "                  5091.      137.     37.2  2.29e-301
##  8 "STATUSCRP - CLASSIFIED REGULAR PART-~  -26370.      621.    -42.4  0
##  9 "STATUSCTF - CLASSIFIED TEMPORARY FUL~  -10222.     1886.     -5.42 5.99e-  8
## 10 "STATUSCTP - CLASSIFIED TEMPORARY FUL~  -28160.    22380.     -1.26 2.08e-  1
## 11 "STATUSCTP - CLASSIFIED TEMPORARY PAR~  -31956.     2940.    -10.9  1.66e- 27
## 12 "STATUSERF - EXEMPT REGULAR FULL-TIME~  102883.      846.    122.   0
## # i 5 more rows
```

Evaluate model

```
texas_train_pred <- predict(texas_fit, texas_train) %>%
  bind_cols(texas_train %>% select(ANNUAL, GENDER))

texas_train_pred
```

Make predictions for training data

```
## # A tibble: 111,054 x 3
##     .pred ANNUAL GENDER
##     <dbl>  <dbl> <chr>
##  1 46469. 28903. "MALE           "
##  2 44658. 56516. "FEMALE         "
##  3 52080. 63000  "FEMALE         "
##  4 44658. 34738. "FEMALE         "
##  5 59585. 30000  "FEMALE         "
##  6 59860. 93480  "FEMALE         "
##  7 49749. 44642. "MALE           "
##  8 57171. 40642. "MALE           "
##  9 52080. 79893. "FEMALE         "
## 10 57171. 95192. "MALE           "
## # i 111,044 more rows
```

R-squared Percentage of variability in salary explained by the model

```
rsq(texas_train_pred, truth = ANNUAL, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rsq     standard       0.196
```

```
rmse(texas_train_pred, truth = ANNUAL, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard      22378.
```

Are models with high or low RMSE are more preferable?

Interpreting RMSE Is this RMSE considered low or high?

```
texas_train %>%
  summarise(min = min(ANNUAL), max = max(ANNUAL))
```

```
##       min    max
## 1 1039.92 450000
```

```
texas_test_pred <- predict(texas_fit, texas_test) %>%
  bind_cols(texas_test %>% select(ANNUAL, GENDER))

texas_test_pred
```

Make predictions for testing data

```
## # A tibble: 37,018 x 3
##     .pred  ANNUAL GENDER
##     <dbl>   <dbl> <chr>
##  1 41378.  39411. "FEMALE         "
##  2 44658.  55130. "FEMALE         "
##  3 46469.  47400  "MALE           "
##  4 30800.  19944  "MALE           "
##  5 30800.   9018  "MALE           "
##  6 63458.  28237. "FEMALE         "
##  7 67007.  96000  "FEMALE         "
##  8 67007. 102000  "FEMALE         "
##  9 56831.  60000  "FEMALE         "
## 10 56831.  30000  "FEMALE         "
## # i 37,008 more rows
```

Evaluate performance on testing data

- RMSE of model fit to testing data

```
rmse(texas_test_pred, truth = ANNUAL, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
```

```
## 1 rmse    standard      22450.
```

- R^2 of model fit to testing data

```r
rsq(texas_test_pred, truth = ANNUAL, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rsq     standard       0.179
```

Training vs. testing

metric	train	test	comparison
RMSE	22378.303	22449.959	RMSE lower for training
R-squared	0.196	0.179	R-squared higher for training

IT Specialists Data Set ## Read in data set

```r
it_specialists <- read.csv("Salaries-of-IT-specialists.csv")

names(it_specialists)[5] <- "Years of Experience"
names(it_specialists)[6] <- "Annual Salary"
names(it_specialists)[10] <- "Employment Type"
names(it_specialists)[11] <- "Number of Projects"
names(it_specialists)[12] <- "Job Satisfaction"

it <- subset(it_specialists, select = -X)
its <- subset(it, select = -ID)

its <- its[1:218, ]

print(unique(its$Gender))
```

```
## [1] "Male"   "Female"
```

```r
itsmales <- sum(its$Gender == "Male")
itsfemales <- sum(its$Gender == "Female")
print(paste("The amount of males in the data set is is:", itsmales))
```

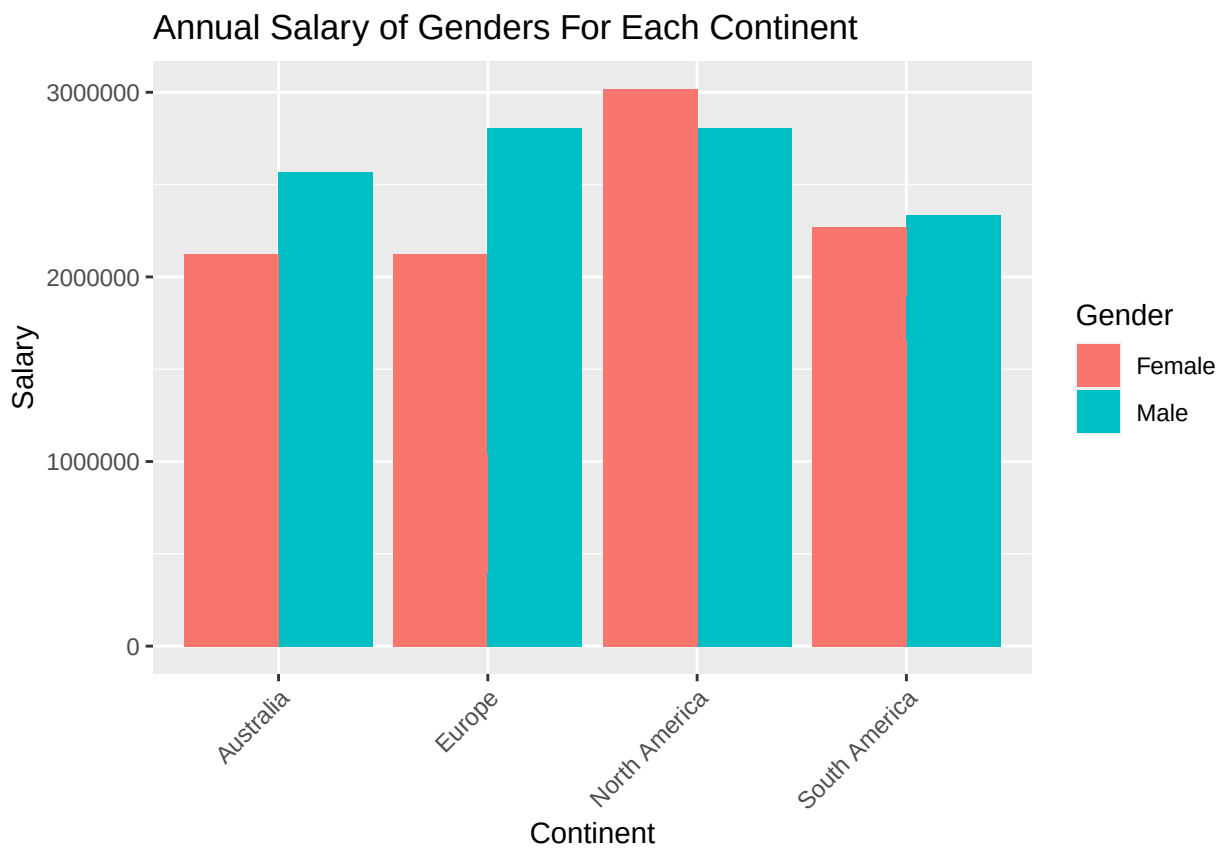
```
## [1] "The amount of males in the data set is is: 158"
```

```r
print(paste("The amount of females in the data set is is:", itsfemales))
```

```
## [1] "The amount of females in the data set is is: 60"
```

Histogram for male and female within each continent

```r
ggplot(its, aes(x = Continent, y = `Annual Salary`, fill = Gender)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Annual Salary of Genders For Each Continent",
       x = "Continent",
       y = "Salary",
       fill = "Gender") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_fill_discrete(name = "Gender")
```



Multiple linear regression using columns as predictors for salary

```
its$Education_numeric <- as.integer(factor(its$Education, levels = c("None", "Bachelor's", "Master's")))
its$Gender_numeric <- as.integer(factor(its$Gender, levels = c("Male", "Female")))

salaries <- lm(`Annual Salary` ~ `Years of Experience`+ Gender_numeric + Age + Education_numeric, its)
names(salaries)
```

```
##  [1] "coefficients"  "residuals"     "effects"       "rank"
##  [5] "fitted.values" "assign"        "qr"            "df.residual"
##  [9] "xlevels"       "call"          "terms"         "model"
```

```
coefficients(salaries)
```

```
##           (Intercept) `Years of Experience`        Gender_numeric
##           1560770.745            -37542.186             82969.743
##                   Age     Education_numeric
##             -2051.367            -68074.293
```

```
summary(salaries)
```

```
##
## Call:
## lm(formula = `Annual Salary` ~ `Years of Experience` + Gender_numeric +
##     Age + Education_numeric, data = its)
##
## Residuals:
```

```
##      Min       1Q   Median       3Q      Max
## -1037370  -400290   -25703   370737  1548442
##
## Coefficients:
##                       Estimate Std. Error t value     Pr(>|t|)
## (Intercept)            1560771     274235   5.691 0.0000000414 ***
## `Years of Experience`   -37542      24275  -1.547        0.123
## Gender_numeric           82970      90900   0.913        0.362
## Age                      -2051       7983  -0.257        0.797
## Education_numeric       -68074      62553  -1.088        0.278
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 575900 on 213 degrees of freedom
## Multiple R-squared:  0.02776,    Adjusted R-squared:  0.009503
## F-statistic:  1.52 on 4 and 213 DF,  p-value: 0.1973
```

Testing / training - see lab 10

- Create an initial split

```
set.seed(1122)
its_split <- initial_split(its) # prop = 3/4 by default
```

- Save training data

```
its_train <- training(its_split)
dim(its_train)
```

```
## [1] 163  13
```

- Save testing data

```
its_test  <- testing(its_split)
dim(its_test)
```

```
## [1] 55 13
```

```
its_mod <- linear_reg() %>%
  set_engine("lm")

its_mod
```

Specify model

```
## Linear Regression Model Specification (regression)
##
## Computational engine: lm
```

```
its_rec <- recipe(`Annual Salary` ~ ., data = its_train) %>%
  # remove zero variance predictors
  step_zv(all_predictors())
```

```
its_rec
```

Build recipe

```
##
## -- Recipe ----------------------------------------------------------------
##
## -- Inputs
## Number of variables by role
## outcome:    1
## predictor: 12
##
## -- Operations
## * Zero variance filter on: all_predictors()
```

```
its_wflow <- workflow() %>%
  add_model(its_mod) %>%
  add_recipe(its_rec)
```

```
its_wflow
```

Build workflow

```
## == Workflow ====================================================================
## Preprocessor: Recipe
## Model: linear_reg()
##
## -- Preprocessor ----------------------------------------------------------------
## 1 Recipe Step
##
## * step_zv()
##
## -- Model -----------------------------------------------------------------------
## Linear Regression Model Specification (regression)
##
## Computational engine: lm
```

```
its_fit <- its_wflow %>%
  fit(data = its_train)
```

```
tidy(its_fit) %>%
  print(n = 12)
```

Fit model

```
## # A tibble: 22 x 5
##    term                   estimate std.error statistic  p.value
##    <chr>                     <dbl>     <dbl>     <dbl>    <dbl>
##  1 (Intercept)            1367291.   478951.     2.85   0.00494
##  2 PositionDeveloper        31860.   152581.     0.209  0.835
##  3 PositionQA_Engineer     -39851.   135480.    -0.294  0.769
##  4 SpecializationDesktop   202763.   194066.     1.04   0.298
```

```
##  5 SpecializationMobile   -120883.   180023.    -0.671  0.503
##  6 SpecializationTesting       NA        NA     NA     NA
##  7 SpecializationWeb           NA        NA     NA     NA
##  8 ContinentEurope         -57121.   223987.    -0.255  0.799
##  9 ContinentNorth America  -55480.   217816.    -0.255  0.799
## 10 ContinentSouth America  -43167.   243901.    -0.177  0.860
## 11 `Years of Experience`   -56321.    30875.    -1.82   0.0702
## 12 GenderMale              -88436.   116272.    -0.761  0.448
## # i 10 more rows
```

Evaluate model

```
its_train_pred <- predict(its_fit, its_train) %>%
  bind_cols(its_train %>% select(`Annual Salary`, Gender))
```

Make predictions for training data

```
## Warning in predict.lm(object = object$fit, newdata = new_data, type =
## "response", : prediction from rank-deficient fit; consider predict(.,
## rankdeficient="NA")
```

```
its_train_pred
```

```
## # A tibble: 163 x 3
##       .pred `Annual Salary` Gender
##       <dbl>           <dbl> <chr>
##  1 1156068.          395436 Male
##  2 1703935.         2334960 Male
##  3 1005644.         1392960 Male
##  4 1407467.         1338795 Female
##  5 1576237.         1752935 Female
##  6 1459384.         1687111 Female
##  7 1344079.          902935 Male
##  8 1267559.          727111 Male
##  9 1180468.          732821 Male
## 10 1337081.         1630396 Male
## # i 153 more rows
```

R-squared Percentage of variability in salary explained by the model

```
rsq(its_train_pred, truth = `Annual Salary`, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rsq     standard       0.102
```

```
rmse(its_train_pred, truth = `Annual Salary`, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard     526189.
```

Are models with high or low RMSE are more preferable?

Interpreting RMSE Is this RMSE considered low or high?

```
its_train %>%
  summarise(min = min(`Annual Salary`), max = max(`Annual Salary`))
```

```
##        min     max
## 1 354110.1 3014686
```

```
its_test_pred <- predict(its_fit, its_test) %>%
  bind_cols(its_test %>% select(`Annual Salary`, Gender))
```

Make predictions for testing data

```
## Warning in predict.lm(object = object$fit, newdata = new_data, type =
## "response", : prediction from rank-deficient fit; consider predict(.,
## rankdeficient="NA")
```

```
its_test_pred
```

```
## # A tibble: 55 x 3
##       .pred `Annual Salary` Gender
##       <dbl>           <dbl> <chr>
##  1 1219199.         1507343 Male
##  2 1347136.         1124939 Male
##  3 1279960.         1630396 Male
##  4 1203086.         1009436 Male
##  5 1121469.          758376 Male
##  6 1022229.         1556575 Male
##  7 1317186.         1541572 Female
##  8 1180646.         1205874 Male
##  9 1267559.          727111 Male
## 10 1197600.          495252 Male
## # i 45 more rows
```

Evaluate performance on testing data

- RMSE of model fit to testing data

```
rmse(its_test_pred, truth =`Annual Salary`, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard     644067.
```

- R^2 of model fit to testing data

```
rsq(its_test_pred, truth = `Annual Salary`, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rsq     standard      0.0141
```

Training vs. testing

metric	train	test	comparison
RMSE	526188.859	644067.334	RMSE lower for training
R-squared	0.102	0.014	R-squared higher for training

^ taken from lab 10**

White House Data Set

```
white_house <- read.csv("wh_staff_dataset.csv")
glimpse(white_house)
```

```
## Rows: 10,600
## Columns: 7
## $ year           <int> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1~
## $ name           <chr> "Abedin,Huma M.", "Abrams,Lori E.", "Alcorn,Brian A.", ~
## $ gender         <chr> "Female", "Female", "Male", "Female", "Female", "Female~
## $ status         <chr> "Employee", "Employee", "Employee", "Employee", "Employ~
## $ salary         <int> 27500, 35000, 25000, 22000, 27624, 47000, 73163, 54629,~
## $ pay_basis      <chr> "Per Annum", "Per Annum", "Per Annum", "Per Annum", "Pe~
## $ position_title <chr> "OFFICE MANAGER", "DEPUTY DIRECTOR, WHITE HOUSE GIFTS",~
```

```
# remove NAs to check for max salary for box plot axis
colSums(is.na(white_house))
```

```
##           year           name         gender         status         salary
##              0              0              0              0              5
##      pay_basis position_title
##              0              0
```

```
wh <- na.omit(white_house)
colSums(is.na(wh))
```

```
##           year           name         gender         status         salary
##              0              0              0              0              0
##      pay_basis position_title
##              0              0
```

```
whhs <- wh[wh$salary != 0, ]

max(whhs$salary)
```

```
## [1] 239595
```

```
# remove data with blank gender entry for boxplot
print(unique(whhs$gender))
```

```
## [1] "Female" "Male"   ""
```

```
wh2 <- whhs[whhs$gender != "", ]

print(unique(wh2$gender))
```

```
## [1] "Female" "Male"
```

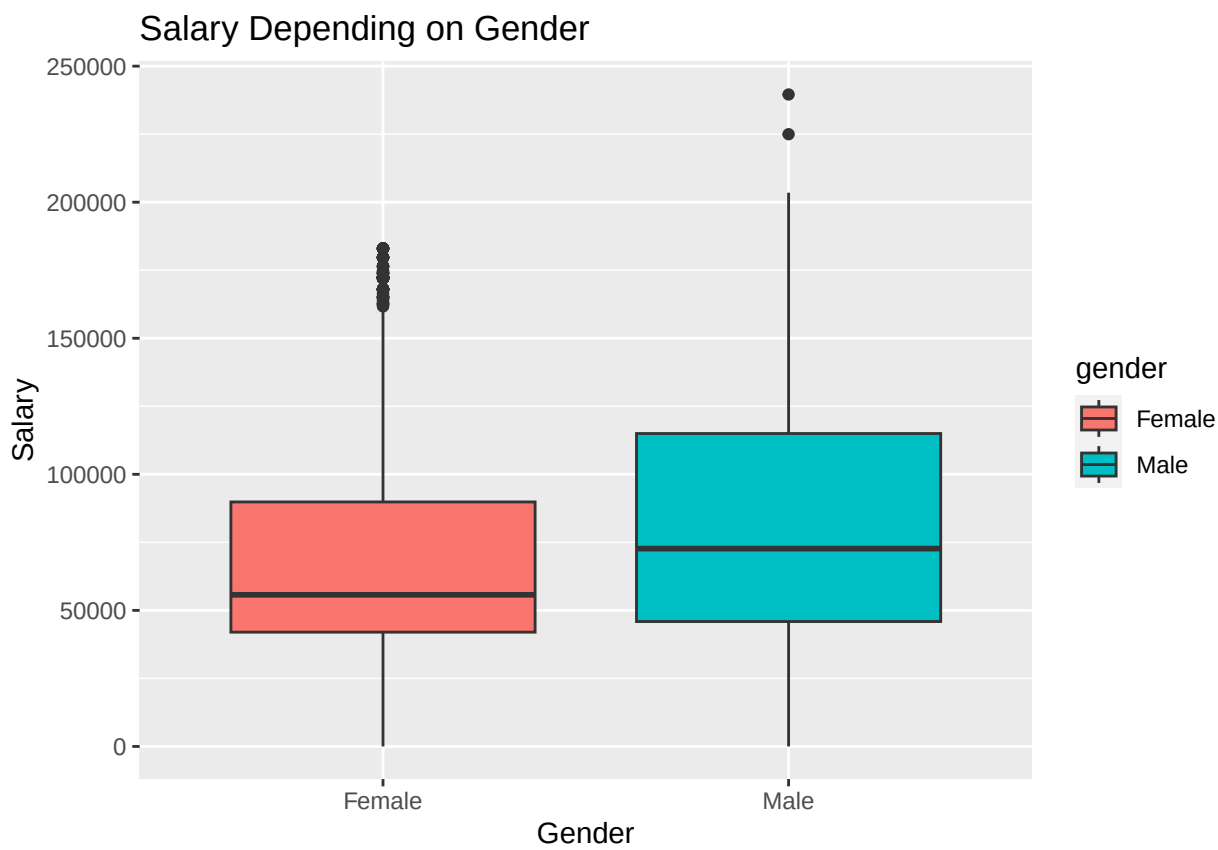
```
whmales <- sum(wh2$gender == "Male")
whfemales <- sum(wh2$gender == "Female")
print(paste("The amount of males in the data set is is:", whmales))
```

```
## [1] "The amount of males in the data set is is: 4978"
```

```r
print(paste("The amount of females in the data set is is:", whfemales))
```

```
## [1] "The amount of females in the data set is is: 5577"
```

```r
options(scipen = 250000)
ggplot(wh2, aes(x = gender, y = salary, fill = gender)) +
  geom_boxplot() +
  labs(title = "Salary Depending on Gender",
       y = "Salary",
       x = "Gender")
```




```r
mean_salaries <- aggregate(salary ~ gender, data = wh2, FUN = mean)

mean_salaries
```

```
##   gender   salary
## 1 Female 68641.35
## 2   Male 83051.63
```

```r
median_salaries <- aggregate(salary ~ gender, data = wh2, FUN = median)

median_salaries
```

```
##   gender salary
## 1 Female  55717
## 2   Male  72700
```

line graph for male and female within each year

```
ggplot(wh2, aes(x = year, y = salary, color = gender)) +
  geom_line() +
  labs(title = "Annual Salary Each Year for Males and Females",
       x = "Year",
       y = "Annual Salary",
       color = "Gender") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_color_discrete(name = "Gender")
```

```
# max salary male or female
max_row <- wh2[which.max(wh2$salary), ]
max_row
```

```
##      year               name gender   status salary pay_basis
## 9850 2019 Czwartacki,John S.   Male Detailee 239595 Per Annum
##                                                       position_title
## 9850 SENIOR ADVISOR TO THE CHIEF OF STAFF FOR STRATEGY AND STAKEHOLDER ENGAGEMENT
```

```
print(unique(wh$status))
```

```
## [1] "Employee"             "Detailee"             "Employee (part-time)"
```

```
print(unique(wh$pay_basis))
```

```
## [1] "Per Annum" ""          "Per Diem"
```

```r
# changing categorical vars

wh2$gender_num <- as.integer(factor(wh2$gender, levels = c("Male", "Female")))
wh2$status_num <- as.integer(factor(wh2$status, levels = c("Employee", "Detailee", "Employee (part-time
wh2$pay_num <- as.integer(factor(wh2$pay_basis, levels = c("Per Annum", "Per Diem")))

# multiple linear regression

salaries2 <- lm(salary ~ gender_num + year + pay_num + status_num, wh2)
names(salaries2)
```

```
##  [1] "coefficients"  "residuals"     "effects"       "rank"
##  [5] "fitted.values" "assign"        "qr"            "df.residual"
##  [9] "xlevels"       "call"          "terms"         "model"
```

```r
coefficients(salaries2)
```

```
##  (Intercept)   gender_num         year      pay_num   status_num
## -3558659.454   -12585.945     1840.651   -73957.816    28695.674
```

```r
summary(salaries2)
```

```
##
## Call:
## lm(formula = salary ~ gender_num + year + pay_num + status_num,
##     data = wh2)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -107006  -27637  -11027   22915  111133
##
## Coefficients:
##                Estimate  Std. Error t value             Pr(>|t|)
## (Intercept) -3558659.45   107167.16 -33.207 < 0.0000000000000002 ***
## gender_num    -12585.94      715.72 -17.585 < 0.0000000000000002 ***
## year            1840.65       52.47  35.079 < 0.0000000000000002 ***
## pay_num       -73957.82    18314.29  -4.038            0.0000542 ***
## status_num     28695.67     1449.51  19.797 < 0.0000000000000002 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 36620 on 10550 degrees of freedom
## Multiple R-squared:  0.1613, Adjusted R-squared:  0.161
## F-statistic: 507.2 on 4 and 10550 DF,  p-value: < 0.00000000000000022
```

Testing / training - see lab 10

- Create an initial split

```r
wh_tt <- wh2[, -(1:2)]
wh_tt <- wh_tt[, -(5:8)]
# take away names, positions (too many to count), year, and num cat vars
set.seed(1122)
wh_split <- initial_split(wh_tt) # prop = 3/4 by default
```

- Save training data

```r
wh_train <- training(wh_split)
dim(wh_train)
```

```
## [1] 7916    4
```

- Save testing data

```r
wh_test  <- testing(wh_split)
dim(wh_test)
```

```
## [1] 2639    4
```

```r
wh_mod <- linear_reg() %>%
  set_engine("lm")

wh_mod
```

Specify model

```
## Linear Regression Model Specification (regression)
##
## Computational engine: lm
```

```r
wh_rec <- recipe(salary ~ ., data = wh_train) %>%
  # remove zero variance predictors
  step_zv(all_predictors())
```

```r
wh_rec
```

Build recipe

```
##
## -- Recipe ----------------------------------------------------------------------
##
## -- Inputs
## Number of variables by role
## outcome:   1
## predictor: 3
##
## -- Operations
## * Zero variance filter on: all_predictors()
```

```r
wh_wflow <- workflow() %>%
  add_model(wh_mod) %>%
  add_recipe(wh_rec)
```

```r
wh_wflow
```

Build workflow

```
## == Workflow ====================================================================
## Preprocessor: Recipe
## Model: linear_reg()
##
## -- Preprocessor ----------------------------------------------------------------
## 1 Recipe Step
##
## * step_zv()
##
## -- Model -----------------------------------------------------------------------
## Linear Regression Model Specification (regression)
##
## Computational engine: lm
```

```
wh_fit <- wh_wflow %>%
  fit(data = wh_train)
```

```
tidy(wh_fit) %>%
  print(n = 12)
```

Fit model

```
## # A tibble: 5 x 5
##   term                       estimate std.error statistic  p.value
##   <chr>                         <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)                  96020.     1755.     54.7  0
## 2 genderMale                   14295.      867.     16.5  4.23e-60
## 3 statusEmployee              -29342.     1759.    -16.7  1.96e-61
## 4 statusEmployee (part-time)  -46020.    38526.     -1.19 2.32e- 1
## 5 pay_basisPer Diem           -76825.    19250.     -3.99 6.64e- 5
```

Evaluate model

```
wh_train_pred <- predict(wh_fit, wh_train) %>%
  bind_cols(wh_train %>% select(salary, gender))

wh_train_pred
```

Make predictions for training data

```
## # A tibble: 7,916 x 3
##     .pred salary gender
##     <dbl>  <int> <chr>
##  1 80973. 130500 Male
##  2 66678.  28000 Female
##  3 66678. 115000 Female
##  4 66678.  75000 Female
##  5 80973.  28000 Male
##  6 80973. 130500 Male
##  7 66678.  72022 Female
##  8 80973. 157000 Male
```

```
##  9 66678. 100904 Female
## 10 80973.  36000 Male
## # i 7,906 more rows
```

R-squared Percentage of variability in salary explained by the model

```r
rsq(wh_train_pred, truth = salary, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rsq     standard      0.0676
```

```r
rmse(wh_train_pred, truth = salary, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard      38474.
```

Are models with high or low RMSE are more preferable?

Interpreting RMSE Is this RMSE considered low or high?

```r
rmse(wh_train_pred, truth = salary, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard      38474.
```

```r
wh_train %>%
  summarise(min = min(salary), max = max(salary))
```

```
##   min    max
## 1   1 239595
```

```r
wh_test_pred <- predict(wh_fit, wh_test) %>%
  bind_cols(wh_test %>% select(salary, gender))

wh_test_pred
```

Make predictions for testing data

```
## # A tibble: 2,639 x 3
##     .pred salary gender
##     <dbl>  <int> <chr>
##  1 80973.  25000 Male
##  2 66678.  22000 Female
##  3 66678.  27964 Female
##  4 80973.  30000 Male
##  5 96020. 108824 Female
##  6 66678.  25000 Female
##  7 80973.  30000 Male
##  8 66678.  25897 Female
##  9 66678.  33792 Female
## 10 80973. 100000 Male
```

```
## # i 2,629 more rows
```

Evaluate performance on testing data

- RMSE of model fit to testing data

```r
rmse(wh_test_pred, truth = salary, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard      39247.
```

- R^2 of model fit to testing data

```r
rsq(wh_test_pred, truth = salary, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rsq     standard      0.0550
```

Training vs. testing

metric	train	test	comparison
RMSE	38473.728	39247.380	RMSE lower for training
R-squared	0.068	0.055	R-squared higher for training